

CSCI 112

Programming with C

LECTURE: 12

J. L. Pach

OUTLINE:

- Overloading operators
- Left & Right shift
- typedef keyword
- Function pointer
- Structures
- Padding

OPERATOR

Overloading in C: Compiler Context Resolution

THE BIG PICTURE

In C++, you can overload operators yourself.

In C, the compiler does it for you—it depends on the TYPE!

> The SAME symbol (`*`, `==`, `+`) has DIFFERENT meanings depending on data types and context.

OPERATOR * - THREE DIFFERENT MEANINGS

Context	Code	Meaning	Compiler Does
Multiply	`a * b`	Multiply two ints	`mul(a, b)`
Dereference	`*ptr`	Get value at address	`getValue(ptr)`
Declaration	`int *p`	Declare pointer type	Allocate 4/8 bytes

Example:

```c
int a = 5, b = 3;
int *ptr = &a;
int result = a * b;       /* Binary operator: multiply */
int value = *ptr;         /* Unary operator: dereference */
/* int *new_ptr;          /* Type declaration: pointer */
```

Compiler sees:

```
Line 1: mul(5, 3)          /* Two ints → MULTIPLY */
Line 2: getValue(ptr)      /* Pointer → DEREFERENCE */
Line 3: allocate_space()   /* int* type → ALLOCATE */
```

OPERATOR == TWO DIFFERENT MEANINGS

Type	Code	Meaning	Compiler Does
int/double	`a == b`	Compare values numerically	`(a - b) == 0`
char*	`str1 == str2`	Compare addresses (WRONG for content!)	Compare memory addresses

Example:

```c
int x = 5, y = 5;
if (x == y)              /* Compare VALUES: 5 == 5 → TRUE */
    printf("Equal\n");     /* Output: Equal */

char *str1 = "Hello";
char *str2 = "Hello";
if (str1 == str2)        /* Compare ADDRESSES, not content! */
    printf("Same string\n");    /* Might be FALSE (different allocations) */

if (str1 == str2)        /* Depends on memory layout */
    printf("Same address\n");   /* Might NOT print! */
```

Compiler sees:

```
int comparison:    (x - y) == 0          ✅ Compare values
char* comparison:  (address_of_str1) == (address_of_str2)  ⚠ Compare addresses!
```

Output:

```
Equal                  /* ints are equal */
(nothing printed)      /* addresses are different */
```

OPERATOR + MULTIPLE MEANINGS

Type	Code	Meaning	Result
int/double	`a + b`	Arithmetic addition	Sum
char*	`ptr + 5`	Pointer arithmetic	Address + (5 × sizeof(type))
int + char*	`5 + ptr`	Pointer arithmetic	Address + (5 × sizeof(type))

Example:

```c
int a = 10, b = 20;
int result = a + b;          /* Add: 30 */

char *str = "Hello";
char *ptr2 = str + 5;        /* Add 5: moves 5 bytes forward */
                             /* ptr2 points to 'o' (5th char) */

int arr[10] = {1,2,3,4,5,6,7,8,9,10};
int *p = &arr[0];
```

Compiler sees:

```
int + int:      add(a, b)                          ✅ Arithmetic
char* + int:    add_address(ptr, 5 × sizeof(char)) ✅ Pointer arithmetic
int* + int:     add_address(p, 3 × sizeof(int))    ✅ Pointer arithmetic
```

THE PATTERN: TYPE-BASED OPERATOR RESOLUTION

COMPILER ALGORITHM:

1. Look at the TYPES of operands
2. Match operator to type signature
3. Execute appropriate operation

Example:

*ptr → TYPE(ptr) == pointer → DEREFERENCE

a * b → TYPE(a) == int, TYPE(b) == int → MULTIPLY

str1 == str2 → TYPE(str1) == char*, TYPE(str2) == char* → COMPARE_ADDRESSES

CONSEQUENCES

for Students

⚠ MISTAKE 1: COMPARING STRINGS

```c
/* WRONG - Compares addresses, not content! */
char *str1 = "Hello";
char *str2 = "Hello";
if (str1 == str2)  /* Compiler: compare addresses (might be false!) */
    printf("Same\n");

/* CORRECT - Use strcmp() */
if (strcmp(str1, str2) == 0)  /* Compare actual content */
    printf("Same\n");
```



⚠ MISTAKE 2: FORGETTING TYPE CONTEXT

```c
/* WRONG - Mixes types confusingly */
int *ptr;
int value = ptr * 5;    /* Is this dereference then multiply? */
                        /* Or pointer arithmetic? */

/* CORRECT - Be explicit with parentheses */
int value = (*ptr) * 5;   /* Dereference first, then multiply */
int value2 = *(ptr + 5);  /* Pointer arithmetic, then dereference */
```



✅CORRECT: LEVERAGE TYPE CONTEXT

```c
/* Pointer arithmetic works naturally */
int arr[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int *ptr = arr;

int *next = ptr + 1;        /* Automatically scales by sizeof(int) */
double *dptr = (double*)arr;
double *dnext = dptr + 1;   /* Automatically scales by sizeof(double) */
```



SUMMARY: SAME SYMBOL, DIFFERENT MEANING

Symbol	Meaning Depends On	Examples
`*`	Operands & Context	`a*b` (multiply), `*ptr` (deref), `int*` (type)
`==`	Types of operands	`5==5` (values), `ptr1==ptr2` (addresses)
`+`	Operand types	`5+3` (add), `ptr+5` (pointer arithmetic)
`++`	Context (prefix/postfix)	`++i` (pre), `i++` (post)
`&`	Context	`&x` (address-of), `a&b` (bitwise AND)

The compiler is ALWAYS doing operator overloading based on types!

KEY INSIGHT

- C doesn't have explicit operator overloading like C++,
- but the compiler handles different meanings of the same operator
- based on **types of operands** and **context**.
- Understanding this is crucial for avoiding bugs!

BITWISE

left shift and right shift

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

Priority	Operator	Description	Associativity	Example	Result
1	(), []	Parentheses; Array subscript	Left-to-right	arr[0] * (x + y)	1
	.	Structure and union member access		point.x	1
	->	Structure and union member access through pointer		ppoint->x	1
2	++, --	Prefix & postfix increment and decrement	Right-to-left	++x; x--; x;	6, 6, 5
	+, -, !, ~	(Unary) plus and minus; Logical NOT and bitwise NOT		y =-y; y =+y; !x; ~x	6, -6,0, -6
	*, & , &&	Indirection (dereference); Address-of; Address-of labels		z = &x; *z;	6422276; 5
	(type), sizeof	Cast, Size-of		(int)3.0f; sizeof(x);	3, 4
3	*, /, %	Multiplication, division, and remainder	Left-to-right	6/2 % 2	1
4	+, -	Addition and subtraction		1 + 2; 3 - 1	3, 2
5	**<<, >>**	**Bitwise left shift and right shift**		**4 << 1; 4 >> 2**	**8, 1**
6	<, <=, >, >=	For relational operators <, > and ≤, ≥ respectively		x<y; x<=y; x>y; x>=y	1, 1, 0 ,0
7	==, !=	For relational = and ≠ respectively		x == y, x != 1	1, 0
8	&	Bitwise AND		7 & 3	3
9	^	Bitwise XOR (exclusive or)		255 ^ 0	255
10	|	Bitwise OR (inclusive or)		7 | 3	7
11	&&	Logical AND		1 && 0	0
12	||	Logical OR		1 || 0	1
13	?:	Ternary conditional	Right-to-left	x = (x > y) ? y : x;	-6
14	=	Simple assignment		x = y;	-6
	+=, -=, *=, /=, %=	Assignment by sum, difference, product, quotient, remainder		x+=1; x-=1; //etc.	6, 5
	<<=, >>=, &=, ^=, |=	Assignment by bitwise left shift, right shift, AND, XOR, OR		3<<=1, 8>>=2 //etc.	6, 2
15	,	Comma	Left-to-right	x = 3, y = 1;	3

BITWISE LEFT SHIFT(<<) AND RIGHT SHIFT(>>)

How does work?

```c
int main()
{
    int a = 4;
    int b = 2;
    int result = a >> b;
    printf("a >> b = %d\n", result);
    result = a << b;
    printf("a << b = %d\n", result);

}
```

Result:

a >> b = ?

a << b = ?

EXAMPLE 1/3

Index	7	6	5	4	3	2	1	0	Result
Value 2^ Idx	128	64	32	16	8	4	2	1	
a	0	0	0	0	0	1	0	0	4
b	0	0	0	0	0	0	1	0	2
a >> b									
a << b									

EXAMPLE 2/3

Index	7	6	5	4	3	2	1	0	Result
Value 2^ Idx	128	64	32	16	8	4	2	1	
a	0	0	0	0	0	1	0	0	4
b	0	0	0	0	0	0	1	0	2
a >> b	0	0	0	0	0	0	0	1	1
a << b									

EXAMPLE 3/3

Index	7	6	5	4	3	2	1	0	Result
Value 2^ Idx	128	64	32	16	8	4	2	1	
a	0	0	0	0	0	1	0	0	4
b	0	0	0	0	0	0	1	0	2
a >> b	0	0	0	0	0	0	0	1	1
a << b	0	0	0	1	0	0	0	0	16

BITWISE LEFT SHIFT(<<) AND RIGHT SHIFT(>>)

How does work?

```c
int main()
{
    int a = 4;
    int b = 2;
    int result = a >> b;
    printf("a >> b = %d\n", result);
    result = a << b;
    printf("a << b = %d\n", result);

}
```



```
Result:
a >> b = 1
a << b = 16
```

BITWISE LEFT & RIGHT SHIFT - SUMMARY 1/2

- The left shift `<<` moves all bits in a number to the left by a specified number of positions.
  → Each shift left multiplies the value by 2.
- The right shift `>>` moves all bits to the right by a specified number of positions.
  → Each shift right divides the value by 2 (for unsigned types).
- Can be combined in **expressions** with arithmetic, logical, and assignment operators:
  - `x = (a << 3) | (b & 0x0F);`

> Shifting beyond the bit width of the type → `undefined behavior`
>
> Right shift of signed values may perform arithmetic or logical shift depending on the compiler.

BITWISE LEFT & RIGHT SHIFT - SUMMARY 2/2

Applications:

- Fast multiplication or division by powers of two
- Bit masking and flag operations
- Extracting or packing bits into specific positions
- Embedded systems, hardware control, and data compression

TYPEDEF

keyword

typedef keyword

- C provides a facility called typedef for creating new data type names.
- typedef can be used with functions and structs etc.

Syntax:

```
typedef <type> Symbolic_name
```

```c
#include <stdio.h>

typedef char * String;
typedef char Letter;

int strCmp( String one, String two ) /* <======  here*/
{
   int i;
   for (i = 0; one[i] != '\0' && two[i] != '\0'; i++)
      if(one[i] > two[i])
         return 1;
      else if(one[i] < two[i])
         return -1;
   return 0;
}

int main(int argc, char *argv[])
{
   String text1 = "Some text\n";        /* <======  here*/
   String text2 = "Some text\n";        /* read only! */
   printf( "%d", strCmp(text1, text2) ); // Result '0'
   Letter letter = 'A';                 /* <======  here*/
   printf( "%c", letter );              // Result 'A'
   return 0;
} /* Result: 0A */
```

FUNCTION

pointer

FOREWORD - SIGNATURE OF FUNCTION

Although the concept of a signature officially emerged in the context of methods/functions in C++, it can be argued that functions in C have their own signatures, meaning a unique identifier composed of:

1. `Function name`: Each function has a unique name used to call it.
2. `Parameter list`: The set of arguments that a function takes, along with their types and order, is crucial for its operation.
3. `Return type`: The data type that a function returns upon completion is also an essential part of its definition.

FUNCTION POINTER

Syntax:

```
return-type function-name (only type of parameter declarations, if any);
```

Example:

```
int (*operation)(int, int);
```



Function pointers are used to store the memory address of a function. For a function pointer to be correctly used, the signature of the function it points to must exactly match the signature of the pointer itself. This means the return type and the list of arguments (including their types and order) must be identical.

Function pointer

```c
/* Function prototypes (declarations)*/
int add(int, int);
int subtract(int, int);

/* Function definitions */
int add(int a, int b)
{
  return a + b;
}
int subtract(int a, int b)
{
  return a - b;
}
```

```c
int x = 5, y = 3;
int (*operation)(int, int);
/* Declare a function pointer that can point to functions  */
/* taking two integers and returning an integer  */

operation = add;
/* Assign the address of the 'add' function to the pointer */
int result = operation(x, y);
/* Call the function through the pointer */
printf("Result of addition: %d\n", result);

operation = subtract;
result = operation(x, y);
printf("Result of subtraction: %d\n", result);
```

```
Result:
Result of addition: 8
Result of subtraction: 2
```

Function pointer with `typdef`

```c
int main()
{
  int x = 5, y = 3;
  int (*peration)(int,int);
  operation = add;
  int result = operation(x, y);
  printf("Result of addition: %d\n", result);
  operation = subtract;
  result = operation(x, y);
  printf("Result of subtraction: %d\n", result);

  return 0;
}
```

```c
/* <======  HERE*/
typedef int (*Operation)(int,int);
int main()
{
  int x = 5, y = 3;
  Operation operation = add;  /* <======  here*/
  int result = operation(x, y);
  printf("Result of addition: %d\n", result);
  operation = subtract;
  result = operation(x, y);
  printf("Result of subtraction: %d\n", result);

  return 0;
}
```

> When you use **typedef** with the syntax for a function pointer, you are not creating any pointer. You are simply defining a `type alias`, which means this type does not exist in main until you actually declare a variable of that type!!!

STRUCTURES

user-defined data types

STRUCTURES 1/3

Syntax:

```
struct symbolic_name1
{
        <statement1>
}<symbolic_name2, ...>;
```

Example:

```
struct Point
{
  int x;
  int y;
};
```

- Structures are `user-defined data types` that group together variables of different data types.
- A structure is a collection of one or more variables, possibly of different types, grouped together under single `symbolic_name` for convenient handling.
- Everything that is in angle brackets `<>` is optional.

STRUCTURES 2/3

```c
struct Point
{
  int x;
  int y;
};
```



```c
struct Point point, * pointPtr;
point.x  = 1;
point.y  = 2;

pointPtr = &point;
pointPtr->x = 5; /* OR */ (*pointPtr).x = 5;
```



- To access members of a structure, we use the dot (`.`) operator. When accessing a member through a pointer, we must use the dereferencing operator (`*`) followed by the member access operator (`.`), enclosed in parentheses: `(*symbolic_name).field` .
- Alternatively, we can use the arrow operator (`->`), which is equivalent to `symbolic_name->field` .

STRUCTURES 3/3

```c
struct Point
{
  int x;
  int y;
} point, * pointPtr; // (global) struct Point point, * pointPtr;
```



```c
point.x  = 1;
point.y  = 2;

pointPtr = &point;
pointPtr->x = 5; /* OR */ (*pointPtr).x = 5;
```



- When defining a struct, after the closing brace `}`, the compiler allows you to automatically create global variables (accessible everywhere) of that struct type. You can declare multiple variables by separating them with commas.
- Additionally, you can create pointers to struct instances using the `*` operator.

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *pointPtr = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

Priority	Operator	Description	Associativity	Example	Result
1	(), []	Parentheses; Array subscript	Left-to-right	arr[0] * (x + y)	1
	.	**Structure and union member access**		**point.x**	1
	->	**Structure and union member access through pointer**		**pointPtr->x**	1
2	++, --	Prefix & postfix increment and decrement	Right-to-left	++x; x--; x;	6, 6, 5
	+, -, !, ~	(Unary) plus and minus; Logical NOT and bitwise NOT		y =-y; y =+y; !x; ~x	6, -6,0, -6
	*, & , &&	Indirection (dereference); Address-of; Address-of labels		z = &x; *z;	6422276; 5
	(type), sizeof	Cast, Size-of		(int)3.0f; sizeof(x);	3, 4
3	*, /, %	Multiplication, division, and remainder	Left-to-right	6/2 % 2	1
4	+, -	Addition and subtraction		1 + 2; 3 - 1	3, 2
5	<<, >>	Bitwise left shift and right shift		4 << 1; 4 >> 2	8, 1
6	<, <=, >, >=	For relational operators <, > and ≤, ≥ respectively		x<y; x<=y; x>y; x>=y	1, 1, 0 ,0
7	==, !=	For relational = and ≠ respectively		x == y, x != 1	1, 0
8	&	Bitwise AND		7 & 3	3
9	^	Bitwise XOR (exclusive or)		255 ^ 0	255
10	|	Bitwise OR (inclusive or)		7 | 3	7
11	&&	Logical AND		1 && 0	0
12	||	Logical OR		1 || 0	1
13	?:	Ternary conditional	Right-to-left	x = (x > y) ? y : x;	-6
14	=	Simple assignment		x = y;	-6
	+=, -=, *=, /=, %=	Assignment by sum, difference, product, quotient, remainder		x+=1; x-=1; //etc.	6, 5
	<<=, >>=, &=, ^=, |=	Assignment by bitwise left shift, right shift, AND, XOR, OR		3<<=1, 8>>=2 //etc.	6, 2
15	,	Comma	Left-to-right	x = 3, y = 1;	3

An example

```c
struct MyStruct
{
    int value;
}*p, s;
/* like struct MyStruct  * sPointer; struct MyStruct myStructure;*/

struct MyStruct  * sPointer; /* global pointer */
struct MyStruct myStructure; /* global variable */

struct MyStruct function( struct MyStruct temp )
{
    temp.value += 5;
    return temp;
}
```

Result:

1 2 2 2 2 0 7

```c
struct MyStruct localStruct; /* local variable */
localStruct.value = 1;
struct MyStruct * localStructPointer; /* local pointer */
localStructPointer = &localStruct;
printf( "%d ", localStruct.value );
localStructPointer->value = 2;
printf( "%d ", localStructPointer->value );
printf( "%d ", (*localStructPointer).value );
/*Equivalent to the previous line*/
p = & s;
(*p).value = 2;
printf( "%d ", s.value );
printf( "%d ", p->value );

printf( "%d ", myStructure.value );
myStructure = function( *localStructPointer );
printf( "%d ", myStructure.value );
```

SUMMARY 1/2

- Structures are `user-defined data types` that group together variables of different data types. They provide a way to create custom data types tailored to specific needs.
- **Components:** Structures consist of members (or fields), which can be of different data types.
- **Member access:**
  - **Direct:** To access members of a structure directly, use the dot operator (`.`).
  - **Indirect (through a pointer):** To access members through a pointer, use either:
    - the dereferencing operator (`*`) followed by the member access operator (`.`), enclosed in parentheses: `(*pointer).member` ,
    - the arrow operator (`->`): `pointer->member` .

SUMMARY 1/2

- **Pointers to structures:** You can create pointers to structures, similar to other data types. This allows dynamic memory allocation for structures and passing them as function arguments.
- **Global vs. local variables:** Structures can be declared as global variables (accessible from anywhere in the program) or local variables (accessible only within a specific block of code). You can also create pointers to structures for more flexible memory management.

TYPEDEF & STRUCT 1/2

Normally, when we define a struct, we can also declare global variables or pointers immediately after the closing brace:

```c
struct Point
{
    int x, y;
} p1, *ptr;
```

- The variables p1 and ptr are global objects of this structure type.

We can also **omit the structure tag name** to make it **anonymous**, preventing multiple instances from being created:

```c
struct
{
    int id;
    float value;
} config;
```

- Here, config is the **only instance** of this unnamed structure type.

TYPEDEF & STRUCT 2/2

```
typedef struct
{
  int x, y;
} Point;
```



```
Point symbolic_name;
```



Alternatively, we can use **typedef** to create an **alias** for a structure type:

- Now we can declare variables as `Point p1;` — no need for the keyword struct.
- The name after the definition (`Point`) is a type alias, not a variable.
- This alias does not prevent creating multiple instances — it simply simplifies the syntax.

TYPEDEF & STRUCT - SUMMARY

Form	Description	Example
`struct Tag { ... } var;`	Defines type + creates variable	`struct Point {int x; int y;} p;`
`struct { ... } var;`	Anonymous struct, single instance	`struct {int x;} s;`
`typedef struct { ... } Alias;`	Creates a reusable type alias	`typedef struct {int x;} Point;`

PADDING

Padding

- **What is padding in structures?**
  - Padding is extra space that a compiler adds to a structure to align its members on specific memory addresses. This alignment is often done to improve memory access performance, especially for data types like `integers` and `floating-point` numbers.
- **Why do compilers add padding?**
  - **Performance:**
    - Most processors are optimized to access memory in chunks (like 4 or 8 bytes). Aligning data on these boundaries can significantly speed up memory access.
  - **Hardware architecture:**
    - Different hardware architectures have specific alignment requirements.

An example - padding

```c
#include <stdio.h>
struct Example1
{
   char c;
   int i;
   short s;
};
struct Example2
{
   short s;
   char c;
   int i;
};
int main(int argc, char *argv[])
{
   printf( "Size of a struct Example is = %d\n", sizeof(struct Example1) );
   printf( "Size of a struct Example is = %d\n", sizeof(struct Example2) );
   struct Example1 example1;
   printf( "Struct Example1:\n" );
   printf( "Address of variable c = %d\n", &example1.c );
   printf( "Address of variable i = %d\n", &example1.i );
   printf( "Address of variable s = %d\n", &example1.s );
   struct Example2 example2;
   printf( "Struct Example2:\n" );
   printf( "Address of variable s = %d\n", &example2.s );
   printf( "Address of variable c = %d\n", &example2.c );
   printf( "Address of variable i = %d\n", &example2.i );
   return 0;
}
```

Result:

Size of a struct Example is = 12

Size of a struct Example is = 8

Struct Example1:

Address of variable c = 6487828

Address of variable i = 64878_32 <---

Address of variable s = 6487836

Struct Example2:

Address of variable s = 6487820

Address of variable c = 64878_22 <---

Address of variable i = 6487824

Summary on padding:

- Even though memory was scarce when the C language was invented and every byte was precious, processors were even slower. Every machine instruction that could be saved while maintaining program functionality sped up the program. Therefore, the padding mechanism is a compromise between memory efficiency and program speed. Instead of calculating the memory address and saving one byte, it was better to perform a simple shift trick to make the variable addresses multiples of two. This is because, instead of multiplying (which is a relatively expensive operation for a processor), a bitwise shift can be used, which is extremely cheap. Thus, padding allows for an increase in the required space for storing a structure, but significantly speeds up access to structure members.
- Another important point is that the `sizeof()` operator does not return the minimum size of the structure, but the actual size after taking into account the padding mechanism. To counteract memory waste, you can change the order of variable declarations in the structure as shown in the example.

THANK

You